

53.1. Air — écrire une musique

`#include <Air>` : `Air THEME = { "D04 12", "==", "M14", ... }`; une mélodie, un pas par élément, que `jouer()` lance.

1. Ce qu'il faut comprendre

Ce que c'est : un **Air** est une mélodie écrite **pas par pas**. Chaque pas est une hauteur et un volume (`"D04 12"`); `"=="` tient la note d'avant ; `"--"` est un **silence**.

La ligne à écrire : `#include <Air>`. Un Air écrit ne joue pas tout seul : c'est la partition, pas le musicien. L'étape suivante, `jouer()`, le fait entendre.

Ce qu'il coûte : deux octets par pas, gravés quand on le joue — un Air que rien ne joue ne grave rien.

Essaie : ajoute `"S0L4"` à la fin : la partition a un pas de plus.

On retrouve Air dans 5 leçons

[54](#), [58](#), [59](#), [123](#), [136](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **507 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         29 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte - attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
CopierTuiles   85 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles        224 o ← CopierTuiles - 14 lettres sur 43
"UNE GAMME ECRITE" 16 o ← texte() - un texte gravé
"PAS ENCORE JOUEE" 16 o ← texte() - un texte gravé
```

Lettres gravées : ACEGIJMNOPRSTU

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`, `GAMME_0`

Fonctions du langage appelées :

```
texte() × 2    22 o sur place, dans main() + EcrireTexte (10 o)
image() × 1    3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <Air> : écrire une musique ----

#include <Air>      // Air : une mélodie, pas par pas
#include <texte>

// La partition : dix pas. Elle est écrite, mais rien ne la joue encore.
```

2. UNE MÉLODIE — GAMME

```
Air GAMME = {
  "D04 12", "RE4 12", "MI4 12", "FA4 12",
  "SOL4 12", "LA4 12", "SI4 12", "D05 12",
  "=", "--",
};
```

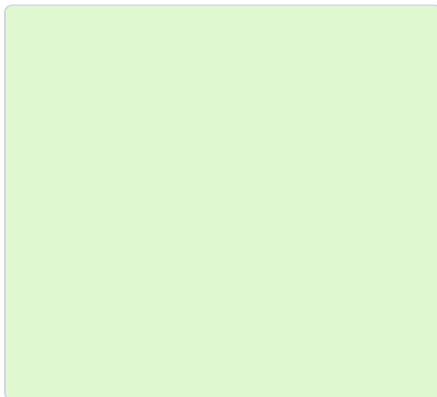
3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  texte(3, 8, "UNE GAMME ECRITE");
  texte(3, 10, "PAS ENCORE JOUEE");

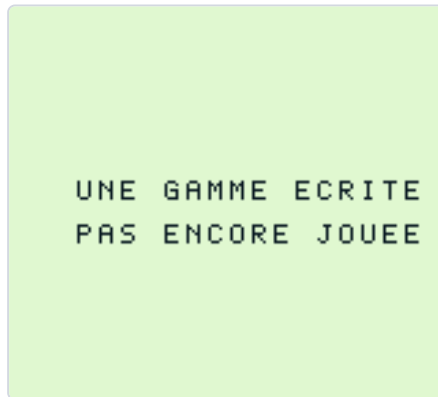
  while (true) {
    image();
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

UNE GAMME ECRITE, PAS ENCORE JOUEE : la partition existe, la console reste muette.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le programme compile avec son Air

✓ et rien ne joue : il faut jouer()

Ensuite : **53.2. La fonction `jouer()` — jouer un air tout seul** — chapitre 8 : Le son. · gameboy3 — écrire une cartouche Game Boy en C++